



程序设计原理

一维数组

任课教师：喻 梅
于瑞国
王建荣
李雪威
赵满坤



Array

Array is a data structure that represents a collection of the same types of data.

```
double myList [10];
```

	myList[0]	5.6	
	myList[1]	4.5	
	myList[2]	3.3	
	myList[3]	13.2	
	myList[4]	4.0	
Array element at index 5	→	myList[5]	34.33 ← Element value
		myList[6]	34.0
		myList[7]	45.45
		myList[8]	99.993
		myList[9]	111.23



Declaring Array Variables

datatype arrayRefVar[arraySize];

Example: double myList[10];

C++ requires that the array size used to declare an array must be a constant expression. For example, the following code is illegal:

int size = 4;

double myList[size]; // **Wrong**

But it would be OK, if size is a constant as follow:

const int size = 4;

double myList[size]; // **Correct**

double list1[10],list2[10];



Declaring Array Variables

```
int n;  
cin >> n;  
double arr[n]; // 某些编译器能通过，但这不是标准 C++
```

//“If the constant-expression (that specifies the bound) is not an integral constant expression, the program is ill-formed.”

Note: Some compilers support variable-length arrays (VLAs) as an extension, but they are not part of the C++ standard.

Tips:

- In standard C++, the array size must be known at compile time.
- To create arrays with runtime size, use `new` or `std::vector`.
- Some compilers allow `double myList[size];` (Variable Length Array), but it's **not portable**.



Accessing Array Elements

Example: `double myList[10];`

The array elements are accessed through the index. Array indices are **0-based**; that is, they start from 0 to `arraySize-1`. For example, the following code adds the value in `myList[0]` and `myList[1]` to `myList[2]`.

```
myList[2] = myList[0] + myList[1];
```

`myList[9]` represents the last element in the array `myList`.



Accessing Array Elementary

➤ No Bound Checking

C++ does not check array's boundary. So, accessing array elements using subscripts beyond the boundary (e.g., myList[-1] and myList[11]) does not does cause syntax errors, but the operating system might report a memory access violation.



Arbitrary Initial Values

When an array is created, its elements are assigned with arbitrary values.

```
#include <iostream>
using namespace std;

int main() {
    int a[5];    // 未初始化
    for (int i = 0; i < 5; i++){
        cout << a[i] << " ";
    }
    return 0;
}
```

32767 0 -4 105 12

-858993460 -858993460 -858993460 -858993460 -858993460



Array Initializers

Declaring, creating, initializing in one step:

dataType arrayName[arraySize] = {value0, value1, ..., valuek};

double myList[4] = {1.9, 2.9, 3.4, 3.5};

This shorthand notation is equivalent to the following statements:

double myList[4];

myList[0] = 1.9;

myList[1] = 2.9;

myList[2] = 3.4;

myList[3] = 3.5;



Array Initializers

Using the shorthand notation, you have to declare, create, and initialize the array all **in one statement**. Splitting it would cause a syntax error. For example, **the following is wrong:**

```
double myList[4];  
myList = {1.9, 2.9, 3.4, 3.5};
```



Array Initializes

Implicit Size

C++ **allows you to omit the array size** when declaring and creating an array using an initializer. For example, the following declaration is fine:

```
double myList[] = {1.9, 2.9, 3.4, 3.5};
```

C++ automatically figures out how many elements are in the array.



Array Initializes

Partial Initialization

C++ allows you to initialize **a part of** the array. For example, the following statement assigns values 1.9, 2.9 to the first two elements of the array. The other two elements will be set to zero. Note that if an array is declared, but not initialized, all its elements will contain “garbage”, like all other local variables.

```
double myList[4] = {1.9, 2.9};
```



Processing Arrays

Initializing Arrays with input value

```
const int ARRAY_SIZE = 10;  
double myList[ARRAY_SIZE];  
  
cout << "Enter" << ARRAY_SIZE << " values:";  
for (int i = 0; i<ARRAY_SIZE;i++)  
    cin >> myList[i];
```



Processing Arrays

Printing arrays

To print an array, you have to print each element in the array using a loop like the following:

```
for (int i = 0; i < ARRAY_SIZE; i++)  
{  
    cout << myList[i] << " ";  
}
```



Processing Arrays

Copying Arrays

Can you copy array using a syntax like this?

list = myList;

This is not allowed in C++. You have to copy individual elements from one array to the other as follows:

```
for (int i = 0; i < ARRAY_SIZE; i++)  
{  
    list[i] = myList[i];  
}
```



Processing Arrays

Summing All Elements

Use a variable named total to store the sum. Initially total is 0. Add each element in the array to total using a loop like this:

```
double total = 0;  
for (int i = 0; i < ARRAY_SIZE; i++)  
{  
    total += myList[i];  
}
```



Processing Arrays

Finding the Largest Element

Use a variable named max to store the largest element. Initially max is myList[0]. To find the largest element in the array myList, compare each element in myList with max, update max if the element is greater than max.

```
double max = myList[0];  
for (int i = 1; i < ARRAY_SIZE; i++)  
{  
    if (myList[i] > max) max = myList[i];  
}
```




Processing Arrays

Finding the **smallest index of the largest element**

```
double max = myList[0];  
int indexOfMax = 0;  
for (int i = 1; i < ARRAY_SIZE; i++)  
{  
    if (myList[i] > max)  
    {  
        max = myList[i];  
        indexOfMax = i;  
    }  
}
```



Example

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int a[10]={1,2,3,4,5,6,7,8,9,10};

    for(int i=0;i<10;i++){
        a[i] = i + 1;
    }

    for(int i=0;i<10;i++){
        cout<<setw(5)<<left<<a[i]<<setw(6)<<left<<a[i]*a[i]<<endl;
    }

    return 0;
}
```

```
1      1
2      4
3      9
4     16
5     25
6     36
7     49
8     64
9     81
10    100
```

Process returned 0 (0x0) execution time : 0.339 s
Press any key to continue.



Exercise

Reversing Numbers

Write a program which reads a sequence and prints it in the reverse order.

Input

The input is given in the following format:

n

$a_1 a_2 \dots a_n$

n is the size of the sequence and a_i is the i th element of the sequence.

Output

Print the reversed sequence in a line. Print a single space character between adjacent elements (Note that your program should not put a space character after the last element).

Constraints

$$n \leq 100$$

$$0 \leq a_i < 1000$$

Sample Input 1

5

1 2 3 4 5

Sample Output 1

5 4 3 2 1

Sample Input 2

8

3 3 4 4 5 8 7 9

Sample Output 2

9 7 8 5 4 4 3 3



Exercise

Finding Missing Cards

Taro is going to play a card game. However, now he has only n cards, even though there should be 52 cards (he has no Jokers).

The 52 cards include 13 ranks of each of the four suits: spade, heart, club and diamond.

Input

In the first line, the number of cards n ($n \leq 52$) is given.

In the following n lines, data of the n cards are given. Each card is given by a pair of a character and an integer which represent its suit and rank respectively. A suit is represented by 'S', 'H', 'C' and 'D' for spades, hearts, clubs and diamonds respectively. A rank is represented by an integer from 1 to 13.

Output

Print the missing cards. The same as the input format, each card should be printed with a character and an integer separated by a space character in a line. Arrange the missing cards in the following priorities:

Print cards of spades, hearts, clubs and diamonds in this order.

If the suits are equal, print cards with lower ranks first.



Exercise

Sample Input

47
S 10
S 11
S 12
S 13
H 1
H 2
S 6
S 7
S 8
S 9
H 6
H 8
H 9
H 10
H 11
H 4
H 5

S 2
S 3
S 4
S 5
H 12
H 13
C 1
C 2
D 1
D 2
D 3
D 4
D 5
D 6
D 7
C 3
C 4
C 5
C 6
C 7

C 8
C 9
C 10
C 11
C 13
D 9
D 10
D 11
D 12
D 13

Sample Output

S 1
H 3
H 7
C 12
D 8